GETTING STARTED TUTORIALS

Written by Robert J. Saunders
And adapted by Rick Vanner

Contents

Starting a project in AppGameKit Studio 2
Commands 2
Remarks 3
Exercise: Commands and Remarks 3
Programming Conventions 4
Loops and Functions 5
Exercise: Loops And Functions 5
Mathematical Operators 8
The Two-Dimensional Grid: X and Y 10
  Exercise: The 2D Grid 10
  Advanced Activity 2 11
Conditional Statements 12
  Exercise: Conditions and Logic 12
Input 13
  Exercise: Input 14
Images And Sprites 14
  Exercise: Images and Sprites 15
Text 16
  Exercise: Text 17
Syntax and Debugging 18
Advanced Variables: Arrays and User-Defined Types 19
  Exercise: Arrays and User-Defined Types 20
Final Thoughts 21

Introduction

Video games have become a multi-billion dollar industry, with yearly gross income to rival that of motion pictures. The creation of video games relies on a wide range of skills, especially science, technology, engineering, and math. Learning to make games is not only fun and educational but can also prepare you for a career in a growing industry!

AppGameKit Studio is the perfect tool to begin learning video game programming (also called "*coding*"). AppGameKit Studio is an integrated development environment (IDE), which means it has a code editor, compiler, debugger, scene editor and more! It uses a very human-readable language with built-in help files. It can create distributable programs for many major platforms.

Starting a project in AppGameKit Studio

When you run AppGameKit Studio for the first time, it will automatically create a blank project for you, and open the code editor. *Project* means all the files that relate to a particular program. This default project can be *compiled* into a running application, and it can act as a basic framework to get started on your first project. The next time you open AppGameKit Studio, your most recent project will be loaded.

The largest area of the screen is the *editor*, and this is where all programming is done. To the left is the *project explorer*, which shows what projects are open at this time, and what files they contain. On top is the *toolbar*, where you have the ability to create, load, and close projects, save files, and to compile, run, and broadcast your work.

Use the buttons in the toolbar to compile and run the default project. It begins by setting the size and title of the window. Then it sets the virtual resolution of the window, and what orientations will be allowed on mobile platforms. Finally, it goes into an endless loop where it prints the current frames per second to the screen.

Commands

Commands are the built-in functional statements your program is made from, and they tell the computer what to do. Commands are *reserved words,* meaning that you can not use their names for anything else. AppGameKit Studio has extensive help files for its commands. To find out what a command does, put your cursor over it and press F1.

The term *syntax* refers to the set of all commands in a computer language, and the proper way to use them. The most basic commands in AppGameKit Studio are just a single word, such as "do", "if", and "exit". More complex commands allow you to pass certain data (called *arguments*) to the commands that will affect what they do. The following command will set the window size of your application on a PC:

```
SetWindowSize(1280,800,0)
```

Remarks

Remarks, also known as *comments*, are lines of code that don't actually perform any actions within your program. Their purpose is to document the intention of a block of code, whether for future reference by another programmer, or to aid you in debugging your own work. The examples in this work are full of comments, and should be helpful and educational.

AppGameKit Studio offers several methods to add comments to your work. Starting a line with the keyword *rem* or a double slash // or the back quote character ` will designate that line as a remark. You can also create block comments by beginning with *remstart* and concluding with *remend*. Anything typed between those two keywords will not be executed.

One further use for remarks is to turn lines of code on and off while debugging. If your program is behaving strangely, you can quickly remove sections of functionality with block comments. Then, if the error disappears, you can look deeper into that particular piece of code.

By now you have probably noticed that different words in the code editor have different colours. Commands are blue, numbers are green, and comments are grey. This is designed to help readability and allow you to quickly skim for the details you're looking for.

Exercise: Commands and Remarks

Objectives
Understand AppGameKit Studio basic functionality
Demonstrate proper syntax when entering commands
Recognize the importance of comments

Materials
AppGameKit Studio
Digital Projector (optional)

Information
Provide you with an overview of the material contained within this chapter

Vocabulary
project, compile, run, editor, toolbar, command, reserved word, syntax, argument, comment, remark

Activity
- Run AppGameKit Studio
- Create a new blank project using the toolbar
- Compile and run the new project
- Change the window size and virtual resolution from 1024 x 768 to 800 x 600
- On the line after SetOrientationAllowed(), type SetPrintSize(64)
- On the line before the first print statement, type Print(GetCurrentTime())

- Add an appropriate comment line above the new print statement, explaining what it does. Use whichever of the comment keywords seems best to you
- Using the built in help files (F1), find a command to print today's date, then add it to your program, with an appropriate comment
- Compile and run (F5) and observe the results. Make any changes that seem necessary and test again

Summary
AppGameKit Studio is easy to use. Commands have a format that must be followed. Comments are useful notes within your programs. Research and experimentation are necessary tools for learning.

Programming Conventions

There are a few basic ideas that should be addressed before going any further. These relate to the way that programs are entered into the editor.

The first convention is *capitalization*. In AppGameKit Studio, capitalization almost never matters. You can type “print”, “PRINT”, or “pRiNt”, and the editor will recognize it as the same command. Capitalization does not matter in naming your variables either, although it is good practice to always enter them the same way, because capitalization does matter in some other programming languages. One special case is the naming of files. On the android platform, your media files must be loaded exactly as they are named, capitalization included.

Next is the idea of “*whitespace*”, which refers to spaces. In general, you can put as many spaces as you want between commands, variables, symbols, and arguments. These examples are valid:

```
SetSpritePosition(1,x,y)
SetSpritePosition  (  1 , x,            y)
```

But you cannot put spaces *within* commands. “Set Sprite Position” is not a command.

Finally, consider *indentation*:

```
Do
      If X < 10
            X = X + 1
      Endif
Loop
```

Indentation serves to show which parts of your programs are *within* other parts. The sample programs created for this guide use single space indentation.

Loops and Functions

Computer programs operate within *loops*. You may have heard the term “60 frames per second”, and this means that the program is doing everything: getting player input, running artificial intelligence routines, and drawing to the screen, 60 times every second. Loops are one of the most fundamental aspects of computer programming.

All programs have a *main loop*, which is where the application begins. All other functions are called from the main loop: loading of images and other assets, displaying the start menu, playing the game, and even shutting down. For our purposes, the main loop is considered to be all code that comes before the first user function.

Computer programs run in a set order, called *control flow*. The first line of code is the first command executed when the program is run, and then the next, and so on. Once your program enters the main loop, it performs the same sequence of instructions over and over, until something tells it to stop.

You may need to use several types of loops: *for* loops, *while* loops, *do* loops, and *repeat* loops. Each one of these is used for a specific purpose. For example, a “for” loop is used to perform a task a certain number of times, and then stop. The “repeat” loop always runs at least once, and will stop when a certain condition is met. By contrast, the “while” loop only executes if a condition is true to begin with, and continues as long as that condition remains true.

A *function* is a block of code that performs a single task, such as checking for collisions between a car and a wall, calculating interest on a virtual loan, or creating a level within your game. In this document, the term *function* will be used to indicate something written by the programmer, while the built in functions of AppGameKit Studio are referred to as *commands*.

If you find that you need to perform a particular action multiple times within your program, it is time to write a function. Functions can perform simple tasks, like calculating and returning the distance between two points in space, or they can represent entire branches of a program, such as the title screen of your app.

Exercise: Loops And Functions

Objectives
Utilize loops to control program flow
Create functions to perform common tasks

Materials
Program Example: “Loops and functions”

Review
Commands, remarks, compile, run

Information
Provide you with an overview of the material contained within this chapter

Vocabulary
loop, main loop, control flow, for, do, while, repeat, function, argument

Activity
- Take 5 minutes to read through the program
- Run the program and interact with it
- Return to the code
- Try to modify the existing behaviours

Questions
1. Why is the function PrintHello() unnecessary, and actually wasteful? (Line 110)
2. Are the other "Print" functions useful? Why?
3. What is the purpose of the repeat loop? (Line 156)
4. If the function PrintHelloTimes() receives '10' as its argument, what will be the value of 't' at the end of the function?
5. Can you find a way to test this?
6. Has anyone discovered how to exit the program with a particular key?
7. Does it work from everywhere in the program?
8. Why or why not?
9. Did anyone notice that there is a "TitleScreen()" function, but it never shows up?
10. Can someone fix that?

Summary
Loops determine the flow of your program. Loops do the same thing repeatedly, until they meet certain exit conditions. Functions should be created to perform any task that your program does more than once.

Variables

Variables are data items that can change their values, and are used extensively in computer programming. A variable can represent the location of an item in your game world, how much health your character has left, or what stage of growth a plant is in. Computer programs exist to manipulate data, and variables are how that is accomplished.

Variables in programming differ somewhat from those found in mathematics. In algebra, variables tend to be a single letter, and represent an unknown quantity. In computing, variable names should be long and descriptive, and their values are declared at the start of the program.

Computer variables have a property called *type*, which determines what sort of data they can hold. *Integer* variables hold integers like 17 and -33. *Floats* hold decimal numbers such as

3.14159. String variables contain letters and words, like names and addresses. Trying to put the wrong kind of data into a variable may result in an error.

Variables also have *scope*, which means the area inside your program where they have meaning. Scope is either *local* or *global*. A variable created inside a function will be local to that function, and will be deleted when that function is complete. A global variable is accessible from anywhere in your program.

To create or *declare* a variable in AppGameKit Studio, you need 3 things: a name, a type, and a scope. Consider the following declaration:

```
Global Health As Integer
```

This tells you that "Health" is an integer variable, and can be accessed and modified from anywhere in your program.

At this point, Health is a valid variable, but it has no value. A numeric variable defaults to zero, while a string variable starts as empty, containing no characters. To give a variable a starting value (to *initialize* it) use the following form: the variable's name, followed by the equals sign (in this case called the *assignment operator*), and then the value you want the variable to have:

```
Health = 100
```

When assigning a value to a string variable, you must always use quotation marks around the data. In the following example, Steve will be the value contained within the string variable Name:

```
Name = "Steve"
```

Exercise: Variables and Scope

Objectives

Understand the use of variables to hold data in computer programs
Apply proper scope to variables in different sections of programs
Distinguish the different types of data that variables can represent

Materials

Program Example: "Variables and Scope"

Review

Loops, functions

Information

Provide you with an overview of the material contained within this chapter

Vocabulary
variable, value, declare, type, integer, float, string, scope, local, global, initialize, assignment operator

Activity

- Run the program
- Fix the "walk function" (hint: the problem is not with the function, but the health variable)
- At line 25, alter the print statement so that the word "Name:" appears before the character's name.
- Fill in the function to reset the character, with Name, Health, and Distance.
- Call the function to reset the character (line 53).
- Allow you to choose their own values for a new name and health.
- Once everything is written for resetting the character, take out the remark character for the line that prints the reset instruction (line 42), so the user can see which key to press.

Questions

1. Why was “Health” not being decreased?
2. When “Health” goes below zero, does the character “die”?
3. Why or why not?

Summary
Variables represent all the data within your program. A variable’s scope determines whether it is stored permanently or deleted. Each variable can only store data of a single type, such as integer, float, or string.

Mathematical Operators

Variables would not be variables without the means to change their values. AppGameKit Studio uses common math notation to operate on variables. In the following example, the variable “Num” is declared and given a value, and then the value is changed:

```
Local num As Integer
Num = 10
Num = Num + 5
Num = Num * 3
```

The final result is 45. Notice what is happening: Num is being redeclared each time, and the value it is given is based on the previous value of Num itself. This is how mathematical operations are performed in AppGameKit Studio. Parentheses can be used to determine precedence in calculations, so using Num = ((Num + 5) * 3) would be valid as well.

Table of Mathematical Operators

+	addition
-	subtraction
*	multiplication
/	division
=	equal to
^	exponent (raised to the power of)
sqrt()	square root
<	is less than
>	is greater than
<>	not equal to
mod	Modulo (remainder after division)

Exercise: Computer Math
Objectives
Demonstrate proper syntax in calculations
Solve problems with computer math

Materials
Program Example: “Computer Math”

Review
Variables

Information
Provide you with an overview of the material contained within this chapter

Activity
- Run the example program and view the results
- Stage 1 demonstrates the order of operations, and looped calculations
- Stage 2 demonstrates float variables, and allows for interaction
- In stage 2, press buttons to perform various operations, and see the results
- Note the slight inaccuracies that occur over time in float variables

- Stage 3 creates and solves a simple algebraic equation
- See if you can verify the results on paper
- Try to modify the program

Challenge

- In stage 2, rewrite the creation of the 5 virtual buttons using a “for” loop
- In stage 3, devise a new algebraic equation and solve it

Summary

Computer programs manipulate data through the use of variables and mathematical operators. While many of these processes are familiar, programming equations requires a specific syntax.

The Two-Dimensional Grid: X and Y

Computer screens are represented as a two-dimensional grid, having width and height. Even when an image appears to be 3D, it is still being rendered as a flat, 2D image. It is a general programming convention to use the variable x to refer to the *width* dimension, and y to indicate *height*.

The upper left corner of the screen is designated as 0,0 , which is also called the *origin*. The lower right corner of the screen will have coordinates equal to the actual width and height of the screen, or the dimensions of your program’s window, if you have set that property.

Taken together, the width and height of the screen are called the *resolution*. The ratio of width to height is called the *aspect ratio*. Some common examples are 4:3 (standard definition) and 16:9 (HDTV). There are many different screen resolutions, especially on mobile devices. This can be one of the most vexing aspects of application development. AppGameKit Studio provides several means of dealing with this problem.

With this simple coordinate system, you can place objects on the screen precisely where you want them. You can also move text and sprites at a particular speed, measure distance, or determine whether two objects are overlapping, all from their “x” and “y”.

The command SetVirtualResolution(x , y) will force your application to have a specific resolution, so that you can design according to known dimensions. The final application will display at that resolution, regardless of the actual screen size of the device you are running on. The downside is that the image may be scaled to fit the screen.

Exercise: The 2D Grid

Objectives

Understand the basic layout of computer screens
Apply this knowledge to achieve desired results

Materials
Program Example: “The 2D Grid Part 1”
Program Example: “The 2D Grid Part 2” (for instructor only)

Information
Provide you with an overview of the material contained within this chapter

Vocabulary
grid, 2D, 3D, dimension, x, y, width, height, origin, resolution, aspect ratio

Demonstration
Using the sample program “The 2D Grid Part 1”, demonstrate the two dimensional grid.

Activity
- Allows you to interact with the program, both the running version and the code
- What is the highest possible value of “X” displayed on the screen, and why is that?
- Change the virtual resolution to 1000 x 750
- Does that change the window size?
- Does it change the maximum values of X and Y?
- Are there any "dead spots" on the screen (places the line can’t reach) ?
- Now change the resolution to 1280 x 800 , and ask those questions again
- Why is that happening?

Advanced Activity 1
(Both of these advanced activities are solved in the example program “The 2D Grid Part 2”.)

Create a new function that will allow the user to change where the line starts, by clicking on the screen. You will need to do the following:
- Add 2 variables to store the new origin coordinates
- Detect the clicking of the mouse
- Alter the DrawLine() command to draw from the new origin, instead of (0,0)

Advanced Activity 2

Create a function that will use the Pythagorean Theorem (A squared + B squared = C squared , a^2 + b^2 = c^2) to calculate the distance between the origin and the end of the line, and print it to the screen.

- This function must receive the x and y of the origin and the end point (4 arguments)
- The function must calculate the distance between those two points
- The function must return the calculated value
- The value returned from the function must be printed to the screen

Summary

The screen of your computer, tablet, and cell phone are all two-dimensional, and these dimensions are represented as “x” and “y”. Understanding this simple concept is necessary to properly display all the visual elements of computer programs and games.

Conditional Statements

The *conditional* statement is a core element of computer programming. In essence, *if* something is true, *then* do something about it. If you are too tired then you pass out. If the player taps the screen then open the menu. Conditionals allow for complex interaction between users and your programs.

Short conditional statements can be written entirely on one line:

```
If x < 10 Then x = x + 1
```

But often you may want to perform more than just one operation for a particular condition. It is also possible that you want to do something in the opposite case as well:

```
If x < 10
      x = x + 1
      Print( “x is less than ten”)
Else
      Print( “x is at least ten!”)
Endif
```

When the program is running and it encounters this block of code, it will only execute one outcome per loop. You will never see “x is less than ten” and “x is at least ten!” at the same time. The program *branches* at that point.

Two new keywords are introduced above: *else* and *endif*. Endif must be placed at the end of a multi-line conditional. Else is used to consider the opposite case.

Logical Operators

Another very important aspect of computer programming is the *logical operator*. If you want to check that two variables have certain values, use the *and* operator. If you want to check that at least one of several conditions is true, use the *or* operator. For example, “If I am in Antarctica or I am standing in a freezer, and I don’t have a coat, then I am cold.”

Exercise: Conditions and Logic

Objectives

Apply logic to control program flow
Structure statements to detect desired conditions

Materials

Program Example: Conditions and Logic
Program Example: Conditions and Logic 2 (for instructor)

Review
Variables, data types

Information
Provide you with an overview of the material contained within this chapter

Vocabulary
Conditional, if, then, else, endif, and, or

Activity
- You should look over the code for the program example, and try to follow the logic and predict the output
- Run the program and observe the results
- Allow you to experiment with the program
- You should then make the following changes:
- Introduce the “Swimsuit” as a piece of clothing
- If the swimsuit is worn in a cold place, set temperature to "Freezing!"
- If the swimsuit is worn on the beach, set temperature to "Just right"
- If the coat is worn on the beach, set temperature to "Roasting"
- Try their own ideas for location, clothing, and temperature.

Challenge
- Create a selection process so that when the program is run, the player can choose their location and clothing, and they are shown the resulting temperature.

Questions
1. What happens if the variable “Location” is set to “antarctica” instead of “Antarctica”?
2. Look up the command “Lower()” in the help files. Could it be used to solve this problem?

Summary
Conditional statements are one of the foundations of computer program structure. Logical operators assist in the testing of combinations of conditions.

Input

For our programs to do anything interesting, the user needs to be able to interact with them. Most of the example programs provided here have accepted some type of user *input*. There are many types of input that can be detected using AppGameKit Studio: the position of the mouse on the screen, the tilt of a mobile device, or any of the sticks and buttons on a gamepad controller.

Many of the input functions in AppGameKit Studio are made to adapt to their environment. For example, the command GetJoystickX() will return a floating-point value between -1 and 1 on any platform. If you are using a laptop with a joystick plugged in, it will read from that. If there is no joystick, it will rely on the keyboard. And if you are using a mobile device, it will automatically create a virtual joystick on the screen.

AppGameKit Studio also offers platform-specific input commands, such as those to access individual keys on a keyboard, mouse buttons, triggers on a joystick, or the GPS on a smartphone.

Exercise: Input

Objectives
Collect user input
Process user input to perform specific goals

Materials
Program Example: “Input Examples“
Joystick / Gamepad (Optional)
Mobile Devices (Optional)

Information
Provide you with an overview of the material contained within this chapter

Activity

- Experiment with as many types of input as your classroom allows
- Can you think of a way to make sure all the texts stay on the screen?
- Can you understand exactly what is happening with the keyboard input, and why it works?
- Using math commands, increase the movement speed of the keyboard input.
- Are there other types of input you could detect?
- What else could you do with it?

Summary
Computer programs require user input to do anything meaningful. AppGameKit Studio detects a wide range of input types over a variety of platforms.

Images and Sprites

Images are the digital pictures you will use in your programs. Images should be of the .png or .jpg formats, and it is preferable if they have “power of 2” dimensions, meaning 8x8, 32x32, 512x512, and so on. Images must be loaded into memory with the LoadImage() command before they can be used. And while AppGameKit Studio normally does not discriminate when it comes to capitalization, the names of images (and other project media) in your loading commands should match the file names exactly.

All images, sounds, and data files for your project must be stored in that project's *media* folder. The media folder is automatically created the first time your project is run, and is stored in the same place as your other project files.

Once an image is loaded into working memory, it can be turned into a *sprite*. A sprite is an image that can be displayed, moved, resized, and rotated, independently of other objects on the screen. All characters, items, and environments in your game will be made of sprites. Here is an example of simple sprite creation:

```
LoadImage(1,"My_Image.png")
CreateSprite(1,1)
SetSpritePosition(1,100,50)
```

First the image is loaded and stored in image *ID* number 1. Then sprite number 1 is created, using image number 1, that was just loaded. Finally, the sprite is positioned on the screen at 100 pixels across (x) and 50 pixels down (y).

Sprites have more properties than are immediately obvious. For example, the sprite has a size, which defaults to the width and height of the image it is created from. It is possible to change the size, with SetSpriteSize(ID , width , height). But you only need to manipulate the properties that are important to you; AppGameKit Studio does the rest in the background.

Transparency (also called *alpha*) is a very important aspect of sprites. While images are all rectangles, you don't want most of your in-game elements to be boxes. AppGameKit Studio has two methods of handling transparency. The first, which is superior, is to use .png images with an alpha transparency layer. In that case, you can easily control which parts of an image are not drawn. To take advantage of transparency, you must use an image-creation tool that supports it.

The other method is to load your images with the flag *bBlackToAlpha* set to 1. This will tell the computer not to draw any pixel in the image that is completely black, defined as RGB(0,0,0). This can cause problems on certain platforms, or if you want black to be part of your images.

Depth is another important property of sprites, and it controls which sprites are drawn on top of others. Lower values of depth are closer to the viewer, higher values are further away.

Exercise: Images and Sprites

Objectives
Create your game world using sprites
Manipulate their properties to suit your needs

Materials
Sample program: "Images and Sprites"

Review
The 2D Grid, Computer Math

Information
Provide you with an overview of the material contained within this chapter

Vocabulary
Image, media, sprite, ID, transparency, depth

Activity
- Allow you to interact with the sample program for a few minutes
- Then examine the code
- Notice the loading of images and creation of sprites
- Review the movement code for the player and the chaser

Questions
1. What happens when the player's sprite moves out of the black area it starts in?
2. Why does this happen, and how can you fix it?
3. What happens when the chaser catches the player?
4. Which sprite is on top, and why?
5. Look at the code that keeps the player's sprite within the screen bounds. If the screen is 1024 across, why is the right boundary 992?

Challenge
- Rewrite the entire MovePlayerSprite() function as a single line.

Summary
Images and sprites are used to create your game world. They can be manipulated to produce a wide range of effects.

Text

Displaying *text* in your programs is also very important. Most of the examples presented so far have used the *print* command, but that is not suited to most uses. You may need to give the player directions, ask a question, or show their score. AppGameKit Studio has many powerful commands for text manipulation. Here is a simple example:

```
CreateText(1,”My First Text Object”)
SetTextPosition(1,400,100)
SetTextSize(1,32)
```

This series of commands will create a large *text object* near the center of the screen. Texts have many properties in common with sprites, like size, position, and depth, and they are manipulated

in much the same way. They also have some parameters unique to text, such as *alignment* (left, center, or right) and *spacing* (how far apart the individual letters in a word are).

Note that text objects contain *string* variables, which are made up of characters, not values. So in order to display a number in a text, we must first convert it to a string, using the *str()* command, as follows:

```
CreateText(1,"")
Local Score As Integer
Score = 0
SetTextString( 1, "Score: " + str( Score ) )
```

This form uses the addition operator on a string, which is acceptable. The only other math-like commands you can use on strings are = ("is equal to") and <> ("not equal to").

Exercise: Text

Objectives
Convey information using text objects
Present text in visually interesting ways

Materials
Program Example: "Texts"

Review
String variables

Information
Provide you with an overview of the material contained within this chapter

Vocabulary
text, print, text object, alignment

Activity

- Run the sample program and observe the results
- Analyze the parts of the code that manipulate text
- Think about what each of the special effects could be used for
- Try and create a text object that displays the current value of the variable "Counter" in the lower right corner of the screen
- Then, using the help files, find another interesting text property, and create a text object that manipulates it.

Summary

Text is a very important part of computer programs, letting the user know what is happening. AppGameKit Studio has a robust text engine, allowing you to create many special effects.

Syntax and Debugging

The terms "*error*" and "*bug*" are used to indicate the failure of your program to behave as you intended. Sometimes bugs are easy to find and fix, such as when a command is spelled incorrectly or you have given it the wrong number or type of arguments. At other times, it will take more mental effort to determine what went wrong.

Bugs can occur at different stages of your program. Simple syntax errors will be reported at *compile time*. In this case, you will receive specific information including a brief description and the line number where the problem was found.

Other errors may happen during the execution of the program (called "*run-time*"). Perhaps you are referencing an image or sound that does not exist. Maybe your program is trying to access part of an array that is outside its bounds. Or it could be that your hero in the game is moving away from where you tap, instead of towards it.

The process of detecting and removing errors from a computer program is called *debugging*, and can sometimes be a challenging exercise in critical thinking.

Exercise: Syntax and Debugging
Objectives
Analyze error reports to correct mistakes
Apply critical thinking to deduce logic errors

Materials
Program Example: “Syntax and Debugging Part 1”
Program Example: “Syntax and Debugging Part 2” (for demonstration)

Information
Provides you with an overview of the material contained within this chapter

Vocabulary
Error, bug, debugging, compile time, runtime

Activity
- Allow you to interact with the working version of the program (part 2)
- Load the program “Syntax and Debugging Part 1”
- Attempt to run the program
- Find and fix the following errors:
- Line 9 : Windows size should be 800 x 600 (hinted at in line 12)
- Line 31 : Should be a remark. Program will not compile.

- Line 53,54 : The addition and subtraction signs have been switched, causing reverse movement on the Y axis.
- Line 66 : should be "For i=1 To 20" , so that we are checking *all* of the sprites.
- Line 74 : Should be Random(0,700) , not 7000. Sprites are being placed far off the screen.
- Line 75 : "Endif" is required to close the conditional.
- Line 82 : Score is never seen because it is outside the loop.

Questions

1. Did you find all of the errors?
2. Did all of the errors prevent the program from running?
3. How did you find the errors at lines 66 and 74?

Summary

Most syntax errors prevent a program from compiling. Other kinds of errors require critical analysis and deductive reasoning to find and fix.

Advanced Variables: Arrays and User-Defined Types

There is another useful kind of variable called an *array*. Arrays are like filing cabinets, where many things of the same kind can be stored efficiently. For example:

```
Dim Names[9] as string
```

This array allows you to store the names of up to 9 entities. The individual names within this array are referred to as *elements*: the first element (Names[1]) of the array might be “Steve”, while the 9th element might be undefined (hold no value).

The first word in the declaration of an array is *dim* and is short for *dimension*; you are telling the array how many dimensions it has. And yes, it is possible to have multi-dimensional arrays, such as Dim Map[10,10]. In the example above, Names is the variable’s name, and the brackets indicate the *bounds*. That variable allows for 9 entries, and if you try to put in a tenth element, you will generate an “*out of bounds*” error. The final word is the type, just like any other variable. By default, arrays are global, unless you specifically declare them to be local.

User-Defined Types, abbreviated *UDT,* allow for the creation of complex data structures. They are comprised of existing data types (integer, string, float), and use *dot notation* to reference their internal elements. For example:

```
Type PlayerCharacter
 Name As String
 Health As Integer
 Speed As Float
Endtype
```

```
Global PC as PlayerCharacter

PC.Name = "Mo"
PC.Health = 100
PC.Speed = 37.5
```

The global variable "PC" now contains 3 data elements, each of different types. It is accessible from anywhere within the program. Note that "PC" does not hold any value itself, and if you try to access it without one of its elements, you will get an error.

Exercise: Arrays and User-Defined Types

Objectives
Use arrays to store similar data
Build data structures to model complex objects

Materials
Program Example: Advanced Variables

Review
Variables

Information
Provides you with an overview of the material contained within this chapter

Vocabulary
array, element, dim, dimension, bounds, out of bounds, dot notation, user-defined type, type, endtype

Activity
- You should run and read through the sample program
- Examine the definitions and usage of the various data structures

Questions
1. Do you always have 10 walls?
2. Why or why not?
3. Can you think of a way to keep the walls from overlapping?
4. Our player is originally given an x,y position of 4,5.
5. What happens if a wall is placed over the player?
6. Can you make sure that walls are not placed at 4,5?
7. Could you instead place the player after the walls, at a location that is definitely not a wall?

Summary
Data management and manipulation is central to computer programs. Arrays and user-defined types help organize data into meaningful structures.

Final Thoughts

Computer programming teaches engineering and critical thinking skills. Video games are a fun and creative way to introduce you to coding. AppGameKit Studio is a powerful tool for making video games.

Video Game Development

The term *video game development* covers the entire scope of making games, and includes research, design, programming, art, sound, music, story, writing, voice acting, database integration, and more. For a fun look at the different aspects of game development, see Liz England's "The Door Problem":
http://www.lizengland.com/blog/2014/04/the-door-problem/

Game development can be done by a single person, a small team, or a huge corporation. It can be done for fun, learning, or as a serious career path.

Gamasutra

One of the best online resources for everything about game development is Gamasutra. It features articles, reviews, tutorials, and blogs about all aspects of making games;
www.gamasutra.com

AppGameKit Studio

For the free trial version of AppGameKit Studio, or to purchase a full copy for your own software development, visit the official website;
https://www.appgamekit.com/studio

Useful Free Software

For powerful image editing software, try paint.net;
http://www.getpaint.net/

To create 8-bit sound effects, try BFXR;
http://www.bfxr.net/

Audacity is a wonderful free sound editor;
http://www.audacityteam.org/